

Problem 1°

1. the log-likelihood function is

$$\ell_{\beta} = \sum_{i=1}^n [y_i \beta^T x_i - \log(1 + \exp(\beta^T x_i))]]$$

$$\nabla \ell_{\beta} = \sum_{i=1}^n \left(y_i x_i - \frac{x_i \exp(\beta^T x_i)}{1 + \exp(\beta^T x_i)} \right)$$

$$= \sum_{i=1}^n (y_i - p_i) x_i = X^T (y - p)$$

$$\nabla^2 \ell_{\beta} = \sum_{i=1}^n -x_i \left(\frac{1}{[1 + \exp(-\beta^T x_i)]^2} \cdot \exp(-\beta^T x_i) \cdot x_i \right)$$

$$= \sum -x_i p_i (1 - p_i) x_i$$

$$= -X^T W X$$

$$\text{where } W = \text{diag}(p_1(1-p_1), \dots, p_n(1-p_n))$$

$\nabla^2 \ell_{\beta}$ is negative for all β in the interior.

Therefore, the log-likelihood function is concave.

2. The log-likelihood function is unconstrained function, so we can also use Adam algorithm to find MLE for logistic regression.

$$L(\beta) = -\ell_{\beta} = \sum_{i=1}^n [\log(1 + e^{\beta^T x_i}) - y_i(\beta^T x_i)]$$

$$\nabla_{\beta} L(\beta) = \sum_{i=1}^n \frac{e^{\beta^T x_i} x_i}{1 + e^{\beta^T x_i}} - y_i x_i$$

$$= X^T(p - y)$$

Adam updating rule :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot \nabla_{\beta} L(\beta) \quad , \beta_1 \text{ is decay rate}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot (\nabla_{\beta} L(\beta))^2 \quad , \beta_2 \text{ is decay rate}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad , \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\beta^{\text{new}} = \beta^{\text{old}} - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad , \alpha \text{ is learning rate}$$

Pros : ① we don't need the Hessian matrix.

② automatically adjust the learning rate for each parameter.

Cons : ① hyperparameter sensitivity.

3.

the estimated odd $\frac{\hat{p}(y=1|x=x_0)}{\hat{p}(y=0|x=x_0)} = \exp(\hat{\beta}^T x)$

If x increase Δ .

$$\frac{\exp[\hat{\beta}^T (x+\Delta)]}{\exp(\hat{\beta}^T x)} = \exp(\hat{\beta}^T \Delta) = \frac{\hat{p}(y=1|x=x+\Delta)}{\hat{p}(y=0|x=x+\Delta)} \cdot \left[\frac{p(y=1|x=x)}{p(y=0|x=x)} \right]^{-1}$$

Therefore, when x increase Δ , the multiplicative change in odds is $\exp(\hat{\beta}^T \Delta)$

If $\hat{\beta} > 0$, it means the event becomes more likely.

If $\hat{\beta} < 0$, it means the event becomes less likely.

4.

Input = D_{train}

S1: Train model

model = LogisticRegress()

model.fit(X_train, y_train)

S2: Obtain Prediction Probabilities

y_predict = model.predict_proba(X_train)[:, 1]

53% Calculate ROC curve

1-specificity, sensitivity, thresholds = roc-curve(y_train, y_predict)

54% Plot

plot(1-specificity, sensitivity)

5.

① LDA needs very normal distribution on X . Logistic Regression no assumption on X .

② In Logistic Regression, we assume $P(Y=1|X) = \frac{\exp(\beta^T X)}{1 + \exp(\beta^T X)}$

6.

$$P(Y=k|X=x) = \frac{P(Y=k, X=x)}{P(X=x)} = \frac{P(X=x|Y=k) \cdot P(Y=k)}{\sum_{i=1}^K P(X=x|Y=i) \cdot P(Y=i)}$$

$P(Y=k)$ is π_k , $P(X=x|Y=k)$ is $f_k(x)$

$$X|Y=k \sim N(\mu_k, \Sigma) \quad f_k(x) = \frac{1}{(\sqrt{2\pi})^p |\Sigma|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_k)^T \Sigma^{-1} (x - \mu_k)\right)$$

$$\text{therefore, } P(Y=k|X=x) = \frac{\pi_k \cdot \frac{1}{(\sqrt{2\pi})^p |\Sigma|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_k)^T \Sigma^{-1} (x - \mu_k)\right)}{\sum_{i=1}^K \pi_i \cdot \frac{1}{(\sqrt{2\pi})^p |\Sigma|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_i)^T \Sigma^{-1} (x - \mu_i)\right)}$$

We can eliminate $\frac{1}{(2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}}$, so the $P(Y=k | X=x) =$

$$\frac{\pi_k \cdot \exp\left(-\frac{1}{2}(x-\mu_k) \Sigma^{-1} (x-\mu_k)^T\right)}{\sum_{i=1}^K \pi_i \cdot \exp\left(-\frac{1}{2}(x-\mu_i) \Sigma^{-1} (x-\mu_i)^T\right)} \quad \text{by removing the term.}$$

We define a function $h(x) = \ln(\pi_k \cdot \exp(-\frac{1}{2}(x-\mu_k) \Sigma^{-1} (x-\mu_k)^T))$

$$= \ln \pi_k + \left[-\frac{1}{2}(x-\mu_k) \Sigma^{-1} (x-\mu_k)^T \right]$$

$$= \ln \pi_k + \left[\left(-\frac{1}{2} x \Sigma^{-1} + \frac{1}{2} \mu_k \Sigma^{-1} \right) (x-\mu_k)^T \right]$$

$$= \ln \pi_k + \left[-\frac{1}{2} x \Sigma^{-1} x^T + \frac{1}{2} x \Sigma^{-1} \mu_k^T + \frac{1}{2} \mu_k \Sigma^{-1} x^T - \frac{1}{2} \mu_k \Sigma^{-1} \mu_k^T \right]$$

For any class k , the $-\frac{1}{2} x \Sigma^{-1} x^T$ is same.

So we can neglect this term, after neglecting we get $J_k(x)$

$$J_k(x) = \ln \pi_k + \mu_k^{-1} \Sigma^{-1} x - \frac{1}{2} \mu_k \Sigma^{-1} \mu_k^T$$

7. If $E(\hat{\Sigma}) = \Sigma$, then $\hat{\Sigma}$ is an unbiased estimator for the population covariance matrix Σ .

the covariance within-class Σ_k :

$$\Sigma_k = \frac{1}{n_k - 1} \sum_{i: y_i = k} (x_i - \hat{\mu}_k) (x_i - \hat{\mu}_k)^T$$

$$E(\Sigma_k) = \Sigma$$

$$\hat{\Sigma} = \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (x_i - \hat{\mu}_k) (x_i - \hat{\mu}_k)^T$$

$$= \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (n_k - 1) \cdot \frac{1}{(n_k - 1)} (x_i - \hat{\mu}_k) (x_i - \hat{\mu}_k)^T$$

$$= \frac{1}{n - K} \sum_{k=1}^K (n_k - 1) \cdot \Sigma_k$$

$$E(\hat{\Sigma}) = \frac{1}{n - K} \sum_{k=1}^K (n_k - 1) \cdot E(\Sigma_k)$$

$$= \frac{1}{n - K} \cdot \sum_{k=1}^K (n_k - 1) \cdot \Sigma$$

We know $\sum_{k=1}^K n_k = n$. Therefore, $E(\hat{\Sigma}) = \Sigma$

8.

$$P(Y=k|X=x) = \frac{P(Y=k, X=x)}{P(X=x)} = \frac{P(X=x|Y=k) \cdot P(Y=k)}{\sum_{i=1}^K P(X=x|Y=i) \cdot P(Y=i)}$$

$P(Y=k)$ is π_k , $P(X=x|Y=k)$ is $f_k(x)$

$$X|Y=k \sim \mathcal{N}(\mu_k, \Sigma_k) \quad f_k(x) = \frac{1}{(\sqrt{2\pi})^{\frac{p}{2}} |\Sigma_k|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right)$$

$$\text{therefore, } P(Y=k|X=x) = \frac{\pi_k \cdot \frac{1}{(\sqrt{2\pi})^{\frac{p}{2}} |\Sigma_k|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right)}{\sum_{i=1}^K \pi_i \cdot \frac{1}{(\sqrt{2\pi})^{\frac{p}{2}} |\Sigma_i|^{\frac{1}{2}}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i)\right)}$$

we can remove $\left[(\sqrt{2\pi})^{\frac{p}{2}}\right]^{\frac{1}{2}}$ this term, after removing

$$\begin{aligned} \text{we define a function } h(x) &= \ln \left[\pi_k \cdot |\Sigma_k|^{-\frac{1}{2}} \cdot \exp\left(-\frac{1}{2} \cdot (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right) \right] \\ &= \ln(\pi_k) + \ln(|\Sigma_k|^{-\frac{1}{2}}) + \left[-\frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) \right] \\ &= \ln(\pi_k) - \frac{1}{2} \ln(|\Sigma_k|) + \left[\left(-\frac{1}{2} x \Sigma_k^{-1} + \frac{1}{2} \mu_k \Sigma_k^{-1}\right)^T (x - \mu_k) \right] \\ &= \ln(\pi_k) - \frac{1}{2} \ln(|\Sigma_k|) + \left[-\frac{1}{2} x \Sigma_k^{-1} x^T + \frac{1}{2} x \Sigma_k^{-1} \mu_k^T \right. \\ &\quad \left. + \frac{1}{2} \mu_k \Sigma_k^{-1} x^T - \frac{1}{2} \mu_k \Sigma_k^{-1} \mu_k^T \right] \\ &= -\frac{1}{2} x \Sigma_k^{-1} x^T + x^T \Sigma_k^{-1} \mu_k + \ln(\pi_k) - \frac{1}{2} \ln(|\Sigma_k|) \\ &\quad - \frac{1}{2} \mu_k \Sigma_k^{-1} \mu_k^T \end{aligned}$$

we can find the last term $-\frac{1}{2} \mu_k \Sigma_k^{-1} \mu_k^T$ is irrelate with x ,

so we can eliminate this term, after removing we get $J_k(x)$

$$J_k(x) = -\frac{1}{2} X \Sigma_k^{-1} X^T + X^T \Sigma_k^{-1} \mu_k + \ln(\pi_k) - \frac{1}{2} \ln(|\Sigma_k|)$$

9. For confusion matrix, we can construct $K \times K$ matrix.

C_{ij} represents samples with true class i predicted as class j

For ROC.

For each class k , treat it as positive and all others are negative. we have one ROC curve per class.

10.

When k increase, the bias decrease and variance increase.

When k decrease, the bias increase and variance decrease.

Problem 2

```
In [1]: import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
import statsmodels.api as sm

In [2]: train_data = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW2/BreastCancer_train.csv')
test_data = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW2/BreastCancer_test.csv')

In [3]: train_data = train_data.dropna()
test_data = test_data.dropna()

In [4]: y_train = train_data['Class'].map({'benign':1, 'malignant':0})
X_train = train_data.drop(columns='Class')
X_train = sm.add_constant(X_train) # Add a constant term to the predictor

In [5]: y_test = test_data['Class'].map({'benign':1, 'malignant':0})
X_test = test_data.drop(columns='Class')
X_test = sm.add_constant(X_test)
```

Problem 2-1

```
In [6]: LR = sm.Logit(y_train, X_train).fit()

Optimization terminated successfully.
      Current function value: 0.068072
      Iterations 10

In [7]: print(LR.summary())
```

```
Logit Regression Results
=====
Dep. Variable:          Class    No. Observations:          392
Model:                  Logit    Df Residuals:          381
Method:                  MLE     Df Model:              10
Date:                   Mon, 13 Oct 2025    Pseudo R-squ.:        0.8945
Time:                   17:47:15    Log-Likelihood:       -26.684
converged:               True     LL-Null:              -253.05
Covariance Type:        nonrobust    LLR p-value:          5.471e-91
=====
              coef    std err          z      P>|z|      [0.025      0.975]
-----
const          9.9222      1.988      4.991      0.000      6.026     13.818
Id          7.554e-07    1.26e-06      0.600      0.548     -1.71e-06    3.22e-06
Cl.thickness   -0.3681      0.214     -1.717      0.086     -0.788      0.052
Cell.size       0.2763      0.277      0.996      0.319     -0.267      0.820
Cell.shape     -0.5516      0.311     -1.773      0.076     -1.161      0.058
Marg.adhesion  -0.3626      0.145     -2.495      0.013     -0.648     -0.078
Epith.c.size   -0.4263      0.268     -1.589      0.112     -0.952      0.100
Bare.nuclei    -0.2780      0.124     -2.249      0.024     -0.520     -0.036
Bl.cromatin    -0.6205      0.266     -2.333      0.020     -1.142     -0.099
Normal.nucleoli -0.1957      0.161     -1.213      0.225     -0.512      0.120
Mitoses        -0.3505      0.415     -0.844      0.399     -1.165      0.464
=====
```

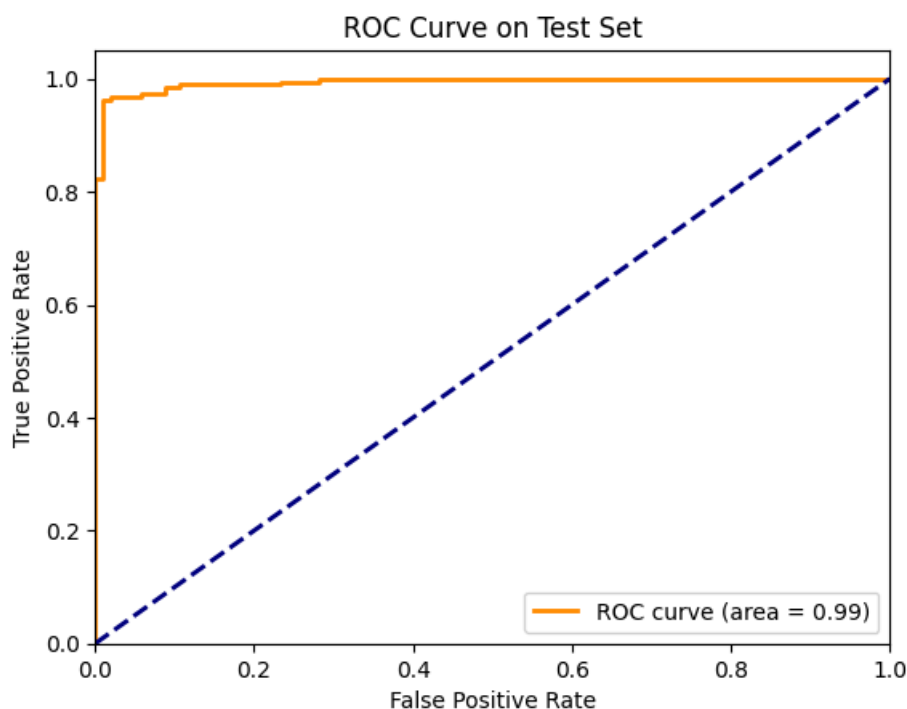
We have used all the predictors to fit a logistic regression model and report the summary.

```
In [8]: y_predict = LR.predict(X_test)

In [9]: from sklearn.metrics import confusion_matrix, roc_curve, auc
import matplotlib.pyplot as plt

In [10]: fpr, tpr, _ = roc_curve(y_test, y_predict)
roc_auc_LR_full = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_LR_full)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve on Test Set')
plt.legend(loc='lower right')
plt.show()
```



We have plotted the ROC curve on the test dataset.

Problem 2-2

```
In [11]: predictors_select = train_data[['Cl.thickness', 'Cell.shape', 'Marg.adhesion', 'Bare.nuclei', 'Bl.cromatin']]
```

```
In [12]: LR_selected = sm.Logit(y_train, predictors_select).fit()
```

```
Optimization terminated successfully.
Current function value: 0.494432
Iterations 7
```

```
In [13]: print(LR_selected.summary())
```

```

=====
                        Logit Regression Results
=====
Dep. Variable:                  Class      No. Observations:                  392
Model:                          Logit      Df Residuals:                    387
Method:                          MLE      Df Model:                        4
Date:                Mon, 13 Oct 2025      Pseudo R-squ.:                   0.2341
Time:                  17:47:15      Log-Likelihood:                   -193.82
converged:                      True      LL-Null:                         -253.05
Covariance Type:      nonrobust      LLR p-value:                      1.138e-24
=====

```

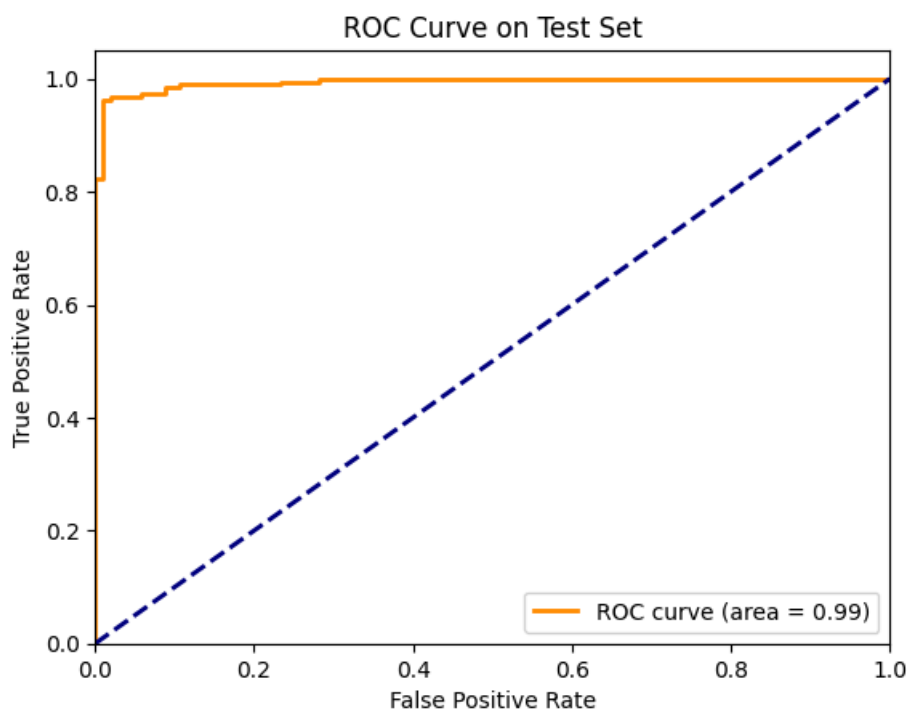
	coef	std err	z	P> z	[0.025	0.975]
Cl.thickness	0.4623	0.069	6.734	0.000	0.328	0.597
Cell.shape	-0.5371	0.095	-5.654	0.000	-0.723	-0.351
Marg.adhesion	-0.1553	0.077	-2.015	0.044	-0.306	-0.004
Bare.nuclei	-0.4088	0.067	-6.106	0.000	-0.540	-0.278
Bl.cromatin	0.3410	0.088	3.890	0.000	0.169	0.513

```
=====
```

```
In [14]: y_predict_selected = LR.predict(X_test)
```

```
In [15]: fpr, tpr, _ = roc_curve(y_test, y_predict_selected)
roc_auc_LR_smaller = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_LR_smaller)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve on Test Set')
plt.legend(loc='lower right')
plt.show()
```



After selecting the predictors, we have fitted the model, reported the summary and plotted the ROC

Problem 2-3

```
In [16]: from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda = LinearDiscriminantAnalysis()

lda.fit(X_train, y_train)
```

```
Out[16]: LinearDiscriminantAnalysis
LinearDiscriminantAnalysis()
```

```
In [17]: # LDA does not have summary like statsmodels, so we could use confusion matrix as summary
lda_train = lda.predict(X_train)

print(confusion_matrix(y_train, lda_train))

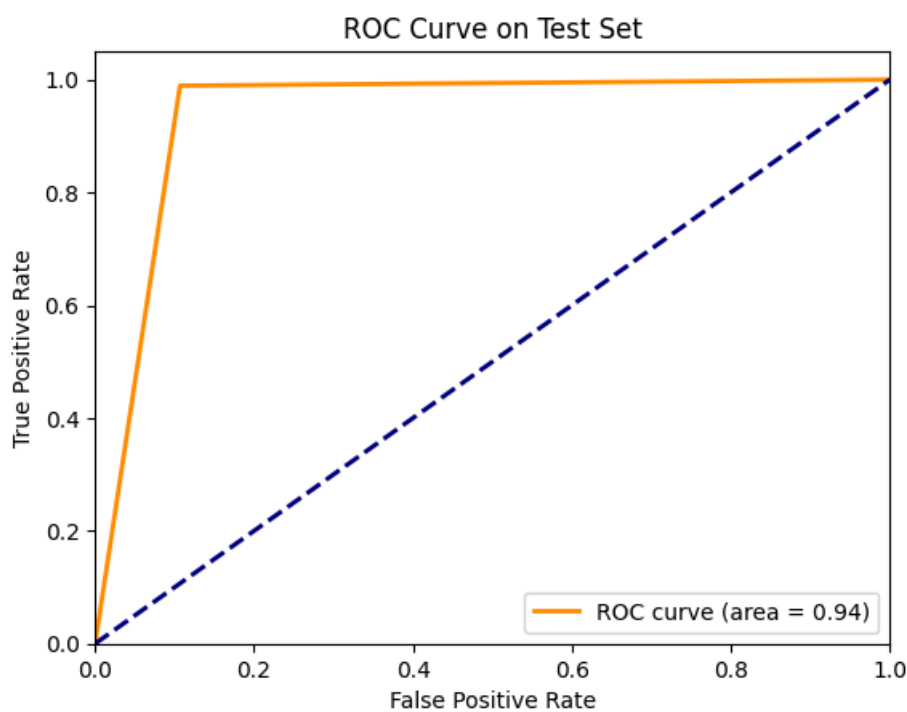
[[128  8]
 [ 6 250]]
```

We have used all the predictors to fit an LDA model and reported the summary

```
In [18]: y_lda_predict = lda.predict(X_test)
```

```
In [19]: fpr, tpr, _ = roc_curve(y_test, y_lda_predict)
roc_auc_LDA_full = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_LDA_full)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve on Test Set')
plt.legend(loc='lower right')
plt.show()
```



We have plotted the ROC curve on the test dataset in LDA.

Problem 2-4

```
In [20]: X_selected_LDA = X_train[['Cl.thickness', 'Cell.shape', 'Marg.adhesion', 'Bare.nuclei', 'Bl.cromatin']]
```

```
In [21]: lda.fit(X_selected_LDA, y_train)

lda_selected_train = lda.predict(X_selected_LDA)

print(confusion_matrix(y_train, lda_selected_train))

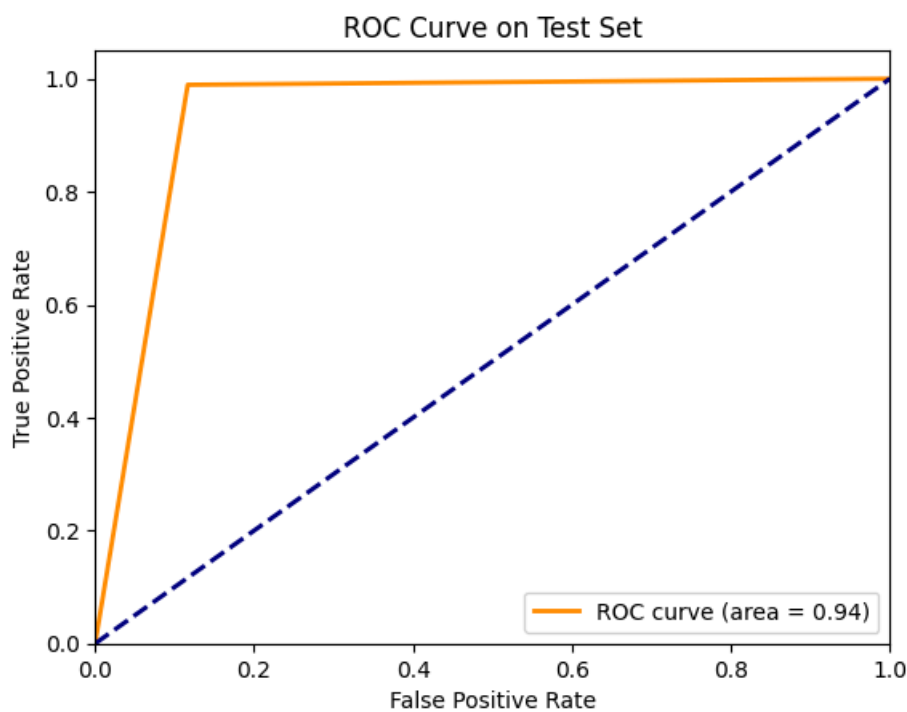
[[125  11]
 [  5 251]]
```

We have used the predictors [Cl.thickness, Cell.shape, Marg.adhesion, Bare.nuclei, Bl.cromatin] to fit an LDA model and reported the summary.

```
In [22]: X_selected_test = X_test[['Cl.thickness', 'Cell.shape', 'Marg.adhesion', 'Bare.nuclei', 'Bl.cromatin']]
y_lda_selected_predict = lda.predict(X_selected_test)
```

```
In [23]: fpr, tpr, _ = roc_curve(y_test, y_lda_selected_predict)
roc_auc_LDA_smaller = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_LDA_smaller)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve on Test Set')
plt.legend(loc='lower right')
plt.show()
```



We have plotted the ROC curve on the test dataset.

Problem 2-5

```
In [24]: from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

roc = QuadraticDiscriminantAnalysis()

y_train = train_data['Class'].map({'benign':1, 'malignant':0})
X_train = train_data.drop(columns='Class')

roc.fit(X_train, y_train)

y_train_predict = roc.predict(X_train)
print(confusion_matrix(y_train, y_train_predict))

[[134  2]
 [ 13 243]]
```

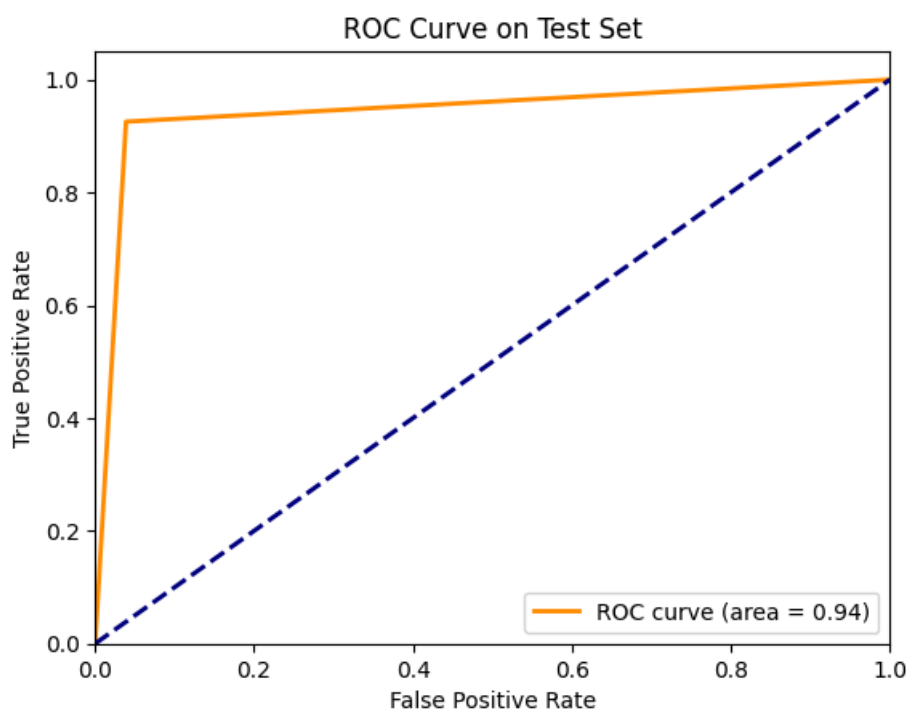
We have used all the predictors to fit a QDA model and reported the summary.

```
In [25]: y_test = test_data['Class'].map({'benign':1, 'malignant':0})
X_test = test_data.drop(columns='Class')

y_pred = roc.predict(X_test)

fpr, tpr, _ = roc_curve(y_test, y_pred)
roc_auc_QDA = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_QDA)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve on Test Set')
plt.legend(loc='lower right')
plt.show()
```



We have plotted the ROC curve on the test dataset.

Problem 2-6

Based on the AUC, we compare all the above models.

```
In [26]: print(f"Logistic Regression (all predictors): AUC = {roc_auc_LR_full:.3f}")
print(f"Logistic Regression (selected predictors): AUC = {roc_auc_LR_smaller:.3f}")
print(f"LDA (all predictors): AUC = {roc_auc_LDA_full:.3f}")
print(f"LDA (selected predictors): AUC = {roc_auc_LDA_smaller:.3f}")
print(f"QDA (all predictors): AUC = {roc_auc_QDA:.3f}")

Logistic Regression (all predictors): AUC = 0.994
Logistic Regression (selected predictors): AUC = 0.994
LDA (all predictors): AUC = 0.941
LDA (selected predictors): AUC = 0.936
QDA (all predictors): AUC = 0.943
```

Problem 3

```
In [27]: data_3 = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW2/cars.csv')
```

```
In [28]: from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.pipeline import make_pipeline
```

```
In [29]: data_3 = data_3.dropna()
```

```
In [30]: X = data_3[['speed']]
y = data_3['dist']
```

Problem 3-1

```
In [31]: loo = LeaveOneOut()

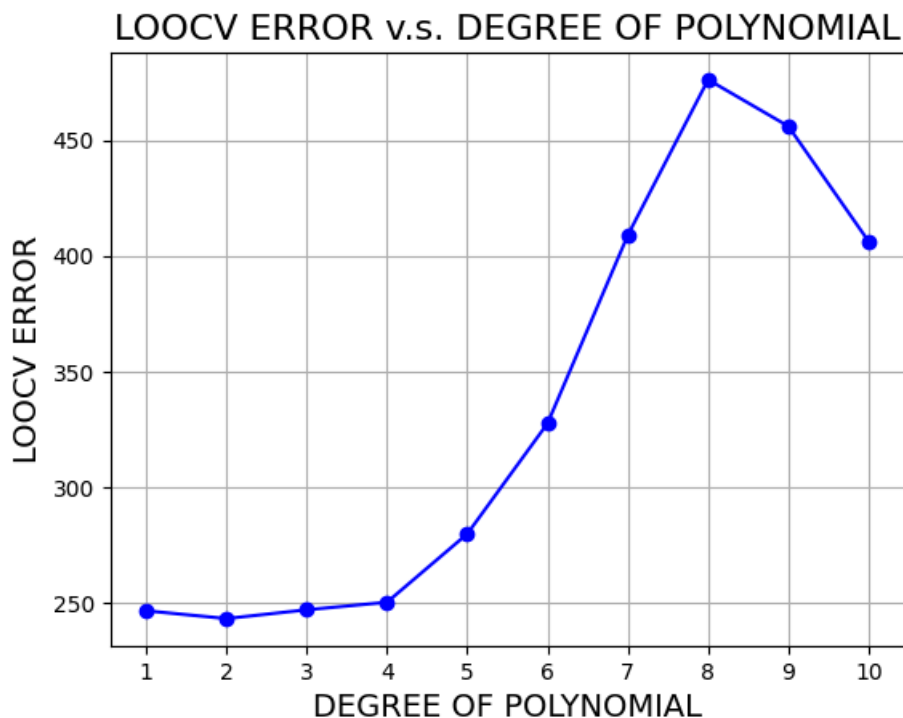
degrees = range(1, 11)
cv_errors_poly = []

for degree in degrees:
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    scores = cross_val_score(model, X, y, cv = loo, scoring = 'neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_errors_poly.append(avg_mse)
    print(f"Degree {degree} :L00_MSE = {avg_mse:.4f}")

plt.figure()
plt.plot(degrees, cv_errors_poly, marker = 'o', linestyle = '-', color = 'b')
```

```
plt.title('LOOCV ERROR v.s. DEGREE OF POLYNOMIAL', fontsize = 16)
plt.xlabel('DEGREE OF POLYNOMIAL', fontsize = 14)
plt.ylabel('LOOCV ERROR', fontsize = 14)
plt.xticks(degrees)
plt.grid(True)
plt.show()
```

```
Degree 1 :L00_MSE = 246.4054
Degree 2 :L00_MSE = 243.0292
Degree 3 :L00_MSE = 246.8288
Degree 4 :L00_MSE = 250.0914
Degree 5 :L00_MSE = 279.6864
Degree 6 :L00_MSE = 327.5014
Degree 7 :L00_MSE = 408.9481
Degree 8 :L00_MSE = 476.2207
Degree 9 :L00_MSE = 456.0527
Degree 10 :L00_MSE = 406.1928
```



According the LOOCV-plot, we can choose degree = 2 as the polynomial model parameter.

Problem 3-2

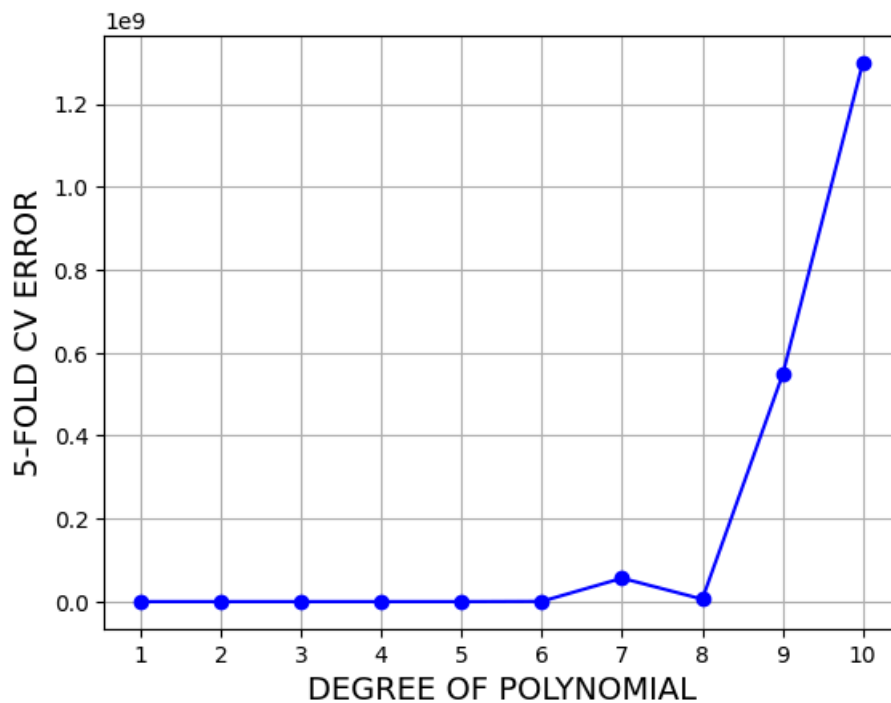
```
In [32]: degrees = range(1, 11)
cv_errors_poly = []

for degree in degrees:
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    scores = cross_val_score(model, X, y, cv = 5, scoring = 'neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_errors_poly.append(avg_mse)
    print(f"Degree {degree} :5-CV_MSE = {avg_mse:.4f}")

plt.figure()
plt.plot(degrees, cv_errors_poly, marker = 'o', linestyle = '--', color = 'b')
plt.title('5-FOLD CV ERROR v.s. DEGREE OF POLYNOMIAL', fontsize = 16)
plt.xlabel('DEGREE OF POLYNOMIAL', fontsize = 14)
plt.ylabel('5-FOLD CV ERROR', fontsize = 14)
plt.xticks(degrees)
plt.grid(True)
plt.show()
```

```
Degree 1 :5-CV_MSE = 265.8523
Degree 2 :5-CV_MSE = 304.5630
Degree 3 :5-CV_MSE = 1879.4172
Degree 4 :5-CV_MSE = 3532.1265
Degree 5 :5-CV_MSE = 23519.1800
Degree 6 :5-CV_MSE = 520279.1559
Degree 7 :5-CV_MSE = 56410050.3495
Degree 8 :5-CV_MSE = 5875146.7085
Degree 9 :5-CV_MSE = 546726877.1205
Degree 10 :5-CV_MSE = 1298927074.1552
```

5-FOLD CV ERROR v.s. DEGREE OF POLYNOMIAL



According the 5-folds CV-plot, we can choose degree = 1 as the polynomial model parameter.

Problem 3-3

```
In [33]: from sklearn.neighbors import KNeighborsRegressor

def gaussian_kernel_smoothing(distance, bandwidth):
    return np.exp(-0.5 * (distance / bandwidth) ** 2)

loo = LeaveOneOut()

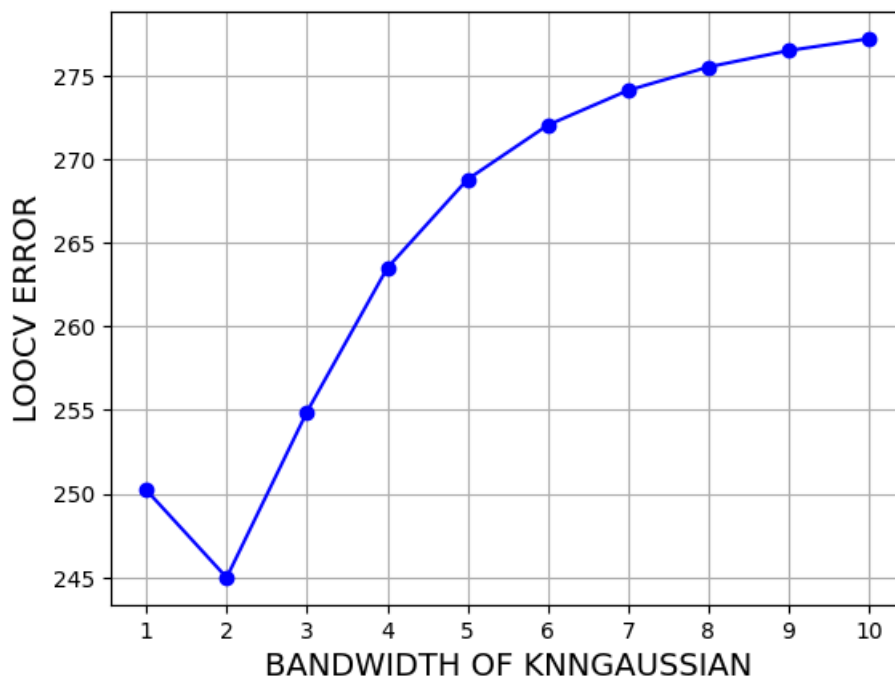
bandwidths = range(1, 11)
cv_errors_knng = []

for bandwidth in bandwidths:
    model = KNeighborsRegressor(n_neighbors = 10, weights = lambda d: gaussian_kernel_smoothing(d, bandwidth))
    scores = cross_val_score(model, X, y, cv = loo, scoring = 'neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_errors_knng.append(avg_mse)
    print(f"Bandwidth {bandwidth} : L00CV_MSE = {avg_mse:.4f}")

plt.figure()
plt.plot(bandwidths, cv_errors_knng, marker = 'o', linestyle = '-', color = 'b')
plt.title('L00CV ERROR v.s. BANDWIDTH OF KNNGAUSSIAN', fontsize = 16)
plt.xlabel('BANDWIDTH OF KNNGAUSSIAN', fontsize = 14)
plt.ylabel('L00CV ERROR', fontsize = 14)
plt.xticks(bandwidths)
plt.grid(True)
plt.show()

Bandwidth 1 : L00CV_MSE = 250.2211
Bandwidth 2 : L00CV_MSE = 244.9927
Bandwidth 3 : L00CV_MSE = 254.8921
Bandwidth 4 : L00CV_MSE = 263.4869
Bandwidth 5 : L00CV_MSE = 268.7843
Bandwidth 6 : L00CV_MSE = 272.0281
Bandwidth 7 : L00CV_MSE = 274.1063
Bandwidth 8 : L00CV_MSE = 275.5030
Bandwidth 9 : L00CV_MSE = 276.4820
Bandwidth 10 : L00CV_MSE = 277.1926
```


LOOCV ERROR v.s. BANDWIDTH OF KNNGAUSSIAN



Based on LOOCV method, we should choose bandwidth = 2.

```
In [34]: from sklearn.neighbors import KNeighborsRegressor

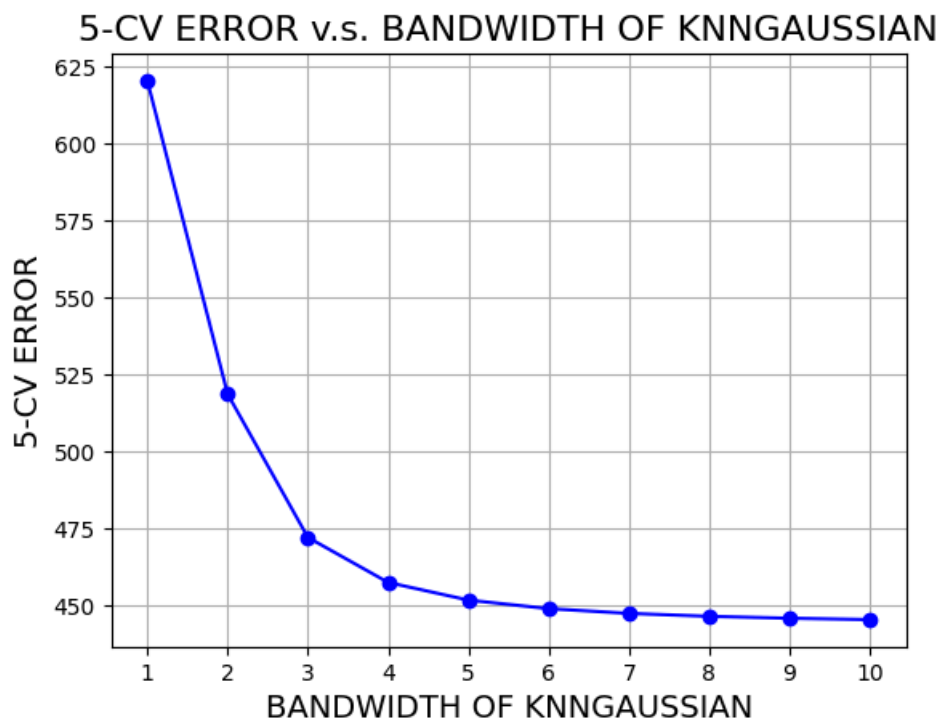
def gaussian_kernel_smoothing(distance, bandwidth):
    return np.exp(-0.5 * (distance / bandwidth) ** 2)

bandwidths = range(1, 11)
cv_errors_knng = []

for h in bandwidths:
    model = KNeighborsRegressor(
        n_neighbors=10,
        weights='lambda', d, bw=h: gaussian_kernel_smoothing(d, bw)
    )
    scores = cross_val_score(model, X, y, cv = 5, scoring = 'neg_mean_squared_error')
    avg_mse = -scores.mean()
    cv_errors_knng.append(avg_mse)
    print(f"Bandwidth {h} :5-CV_MSE = {avg_mse:.4f}")

plt.figure()
plt.plot(bandwidths, cv_errors_knng, marker = 'o', linestyle = '-', color = 'b')
plt.title('5-CV ERROR v.s. BANDWIDTH OF KNNGAUSSIAN', fontsize = 16)
plt.xlabel('BANDWIDTH OF KNNGAUSSIAN', fontsize = 14)
plt.ylabel('5-CV ERROR', fontsize = 14)
plt.xticks(bandwidths)
plt.grid(True)
plt.show()

Bandwidth 1 :5-CV_MSE = 620.6065
Bandwidth 2 :5-CV_MSE = 518.7023
Bandwidth 3 :5-CV_MSE = 472.0541
Bandwidth 4 :5-CV_MSE = 457.4079
Bandwidth 5 :5-CV_MSE = 451.6622
Bandwidth 6 :5-CV_MSE = 448.8878
Bandwidth 7 :5-CV_MSE = 447.3431
Bandwidth 8 :5-CV_MSE = 446.3941
Bandwidth 9 :5-CV_MSE = 445.7684
Bandwidth 10 :5-CV_MSE = 445.3334
```



Based on LOOCV method, we should choose bandwidth = 4.

Problem 3-4

- Based on the LOOCV results, the polynomial regression with degree = 2 with LOO MSE = 243.03 performs better than the KNN–Gaussian kernel model with bandwidth = 2 with LOO MSE = 244.99.
- Under the 5-fold cross-validation, the polynomial regression with degree = 1 achieves the lowest MSE 265.8 and clearly outperforms the KNN–Gaussian model with best MSE 445.

Problem 4

```
In [35]: import pandas as pd
import numpy as np
import statsmodels.api as sm
```

```
In [36]: data_4 = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW2/titanic.csv')
data_4 = data_4.drop(columns='Cabin')
data_4 = data_4.dropna()
data_4
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S
...	...	...	...	...	...	...	...	...	...	...	...
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	Q
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	Q

712 rows x 11 columns

```
In [37]: data_4['Pclass']
# we find the type of Pclass is int64, we should separate the 1 2 3, so we convert the type of Pclass

data_4['Pclass'] = data_4['Pclass'].astype('category')

In [38]: # because we should report the estimated coefficients for Sex/male and pclass/3rd, therefore we use get_dummies
X = pd.get_dummies(data_4[['Pclass','Sex','Age','SibSp','Fare']],drop_first = True)

X = sm.add_constant(X)

X = X.astype(int)

y = data_4['Survived']

X
```

```
Out[38]:
```

	const	Age	SibSp	Fare	Pclass_2	Pclass_3	Sex_male
0	1	22	1	7	0	1	1
1	1	38	1	71	0	0	0
2	1	26	0	7	0	1	0
3	1	35	1	53	0	0	0
4	1	35	0	8	0	1	1
...	...	...	...	...	...	...	...
885	1	39	0	29	0	1	0
886	1	27	0	13	1	0	1
887	1	19	0	30	0	0	0
889	1	26	0	30	0	0	1
890	1	32	0	7	0	1	1

712 rows × 7 columns

```
In [39]: logitmodel = sm.Logit(y,X).fit()
print(logitmodel.summary())
```

Optimization terminated successfully.

Current function value: 0.446245

Iterations 6

Logit Regression Results

```
=====
Dep. Variable:          Survived      No. Observations:          712
Model:                Logit          Df Residuals:              705
Method:                MLE           Df Model:                  6
Date:                  Mon, 13 Oct 2025    Pseudo R-squ.:            0.3387
Time:                  17:47:17           Log-Likelihood:           -317.73
converged:              True            LL-Null:                  -480.45
Covariance Type:        nonrobust        LLR p-value:              2.861e-67
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	4.1643	0.501	8.308	0.000	3.182	5.147
Age	-0.0443	0.008	-5.362	0.000	-0.060	-0.028
SibSp	-0.3917	0.123	-3.182	0.001	-0.633	-0.150
Fare	0.0017	0.002	0.706	0.480	-0.003	0.007
Pclass_2	-1.3024	0.320	-4.075	0.000	-1.929	-0.676
Pclass_3	-2.5197	0.335	-7.530	0.000	-3.176	-1.864
Sex_male	-2.6082	0.216	-12.094	0.000	-3.031	-2.185

Problem 4-1

```
In [40]: coe_sexmale = logitmodel.params['Sex_male']
coe_pclass3 = logitmodel.params['Pclass_3']

conf_int_sexmale = logitmodel.conf_int().loc['Sex_male']
conf_int_pclass3 = logitmodel.conf_int().loc['Pclass_3']
```

```
In [41]: print(f"the estimated coefficients for Sex/male is {coe_sexmale}, 95% confidence intervals is ({conf_int_sexmale[0], conf_int_sexmale[1]})")
print(f"the estimated coefficients for pclass/3rd is {coe_pclass3}, 95% confidence intervals is ({conf_int_pclass3[0], conf_int_pclass3[1]})")
```

the estimated coefficients for Sex/male is -2.6081823852970647, 95% confidence intervals is (-3.0309,-2.1855).
the estimated coefficients for pclass/3rd is -2.5197227240258817, 95% confidence intervals is (-3.1756,-1.8639).

We have reported the estimated coefficients for Sex/male and pclass/3rd, and report their 95% confidence intervals.

Problem 4-2

```
In [42]: from sklearn.utils import resample

n_bootstraps = 1000
bootstrap_coefs = []

# because we should report the estimated coefficients for Sex/male and pclass/3rd, therefore we use get_dummies
X = pd.get_dummies(data_4[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first = True)

X = sm.add_constant(X)
X = X.astype(float)
y = data_4['Survived']

fail_count = 0

np.random.seed(5054244)

for i in range(n_bootstraps):
    bootstrap_sample = resample(data_4, replace = True, n_samples = len(data_4))
    X_bootstrap = pd.get_dummies(bootstrap_sample[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first = True)

    X_bootstrap = sm.add_constant(X_bootstrap)
    X_bootstrap = X_bootstrap.astype(float)
    y_bootstrap = bootstrap_sample['Survived']

    X_bootstrap = X_bootstrap.reindex(columns=X.columns, fill_value=0)

    try:
        bootslogitmodel = sm.Logit(y_bootstrap, X_bootstrap).fit(dis=0)
        bootstrap_coefs.append(bootslogitmodel.params[['Sex_male', 'Pclass_3']])
    except np.linalg.LinAlgError:
        fail_count += 1
        continue

bootstrap_coefs_df = pd.DataFrame(bootstrap_coefs)

ci_bootstrap = bootstrap_coefs_df.quantile([0.025, 0.975])
```

```
In [43]: ci_bootstrap
```

```
Out[43]:
```

	Sex_male	Pclass_3
0.025	-3.054985	-3.300805
0.975	-2.259527	-1.933020

The 95% confidence intervals obtained from Bootstrap are generally wider than those from the original model. This phenomenon reflect greater uncertainty estimated from resampling.

Problem 4-3

```
In [44]: data = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW2/titanic.csv')

print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   PassengerId           891 non-null    int64
1   Survived              891 non-null    int64
2   Pclass               891 non-null    int64
3   Name                 891 non-null    object
4   Sex                 891 non-null    object
5   Age                 714 non-null    float64
6   SibSp              891 non-null    int64
7   Parch             891 non-null    int64
8   Ticket             891 non-null    object
9   Fare              891 non-null    float64
10  Cabin             204 non-null    object
11  Embarked          889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

```
In [45]: print(data.describe())
```

```
count    PassengerId      Survived  Pclass      Age      SibSp  \
count    891.000000    891.000000    891.000000    714.000000    891.000000
mean      446.000000      0.383838      2.308642     29.699118      0.523008
std       257.353842      0.486592      0.836071     14.526497      1.102743
min         1.000000      0.000000      1.000000      0.420000      0.000000
25%       223.500000      0.000000      2.000000     20.125000      0.000000
50%       446.000000      0.000000      3.000000     28.000000      0.000000
75%       668.500000      1.000000      3.000000     38.000000      1.000000
max       891.000000      1.000000      3.000000     80.000000      8.000000

count    Parch      Fare
count    891.000000    891.000000
mean       0.381594     32.204208
std        0.806057     49.693429
min         0.000000      0.000000
25%         0.000000      7.910400
50%         0.000000     14.454200
75%         0.000000     31.000000
max         6.000000    512.329200
```

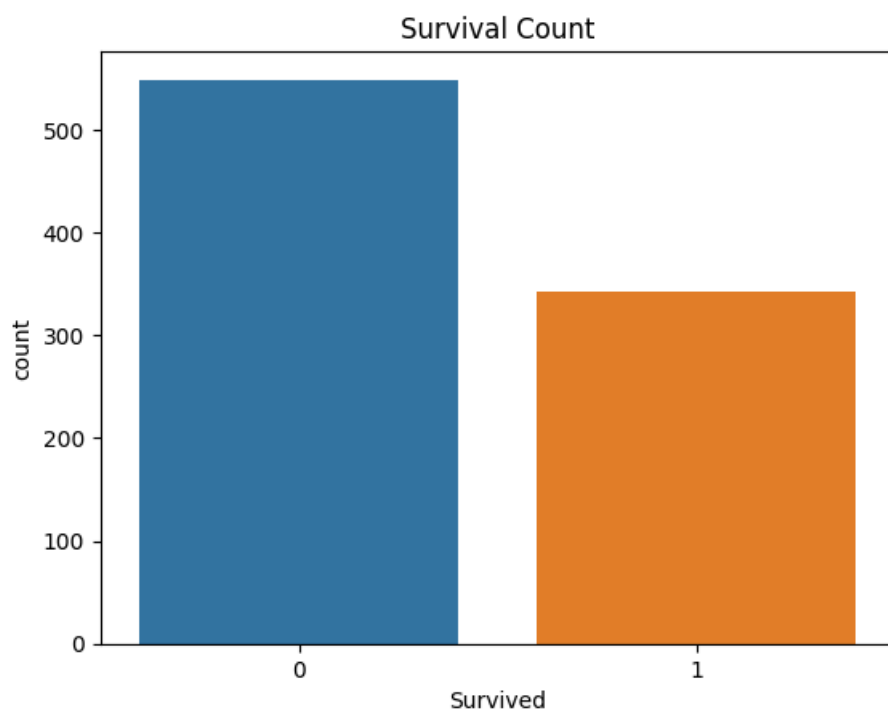
```
In [46]: print(data.isnull().sum())
```

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked         2
dtype: int64
```

```
In [47]: import seaborn as sns
import matplotlib.pyplot as plt

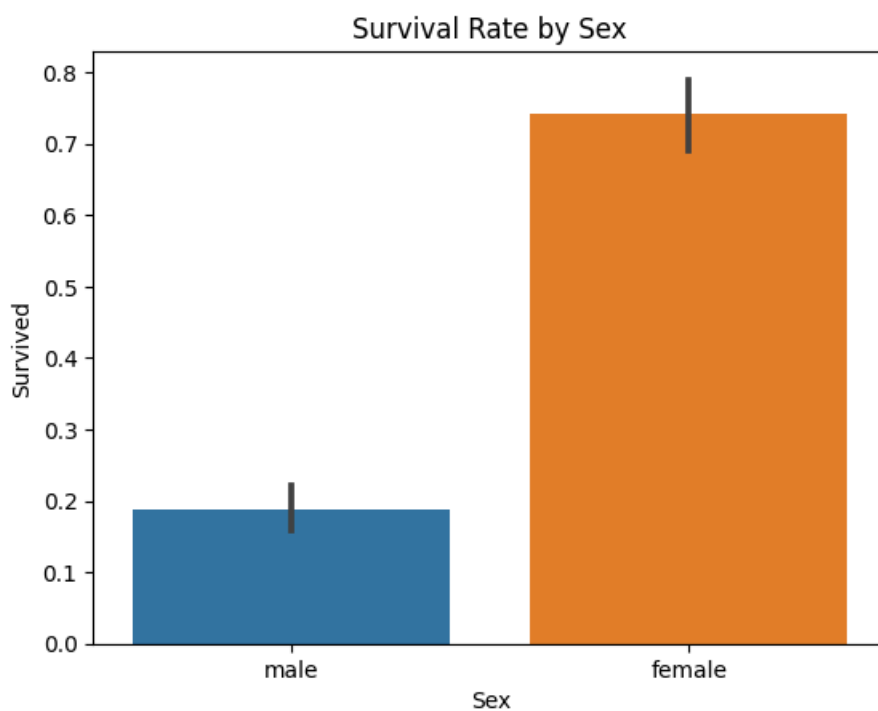
sns.countplot(x='Survived', data=data)
plt.title('Survival Count')
plt.show()

print(data['Survived'].value_counts(normalize=True))
```



```
Survived
0    0.616162
1    0.383838
Name: proportion, dtype: float64
```

```
In [48]: sns.barplot(x='Sex', y='Survived', data=data)
plt.title('Survival Rate by Sex')
plt.show()
```



We can find:

- The overall survival rate is approximately 38%
- The number of deaths is greater than the number of survivors
- The survival rate of women is significantly higher than that of men, reflecting the "women and children first" policy

Problem 4-4

```
In [49]: test_point = data_4.iloc[[0]]
train_data = data_4.iloc[1:]
```

```
In [50]: X_train = pd.get_dummies(train_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
X_train = sm.add_constant(X_train)
X_train = X_train.astype(float)
y_train = train_data['Survived']
```

```
X_test = pd.get_dummies(test_point[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
X_test = sm.add_constant(X_test)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
X_test = X_test.astype(float)
```

```
logit_model = sm.Logit(y_train, X_train).fit()
pred_prob = logit_model.predict(X_test)
```

```
print(f"Predicted survival probability for test point: {pred_prob.iloc[0]:.4f}")
```

Optimization terminated successfully.

Current function value: 0.446778

Iterations 6

Predicted survival probability for test point: 0.0204

```
In [51]: n_bootstraps = 1000
bootstrap_probs = []

for i in range(n_bootstraps):
    bootstrap_sample = resample(train_data, replace=True, n_samples=len(train_data))

    X_boot = pd.get_dummies(bootstrap_sample[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
    X_boot = sm.add_constant(X_boot)
    X_boot = X_boot.reindex(columns=X_train.columns, fill_value=0)
    X_boot = X_boot.astype(float)
    y_boot = bootstrap_sample['Survived']

    try:
        boot_model = sm.Logit(y_boot, X_boot).fit(dis=0, maxiter=100)
        prob = boot_model.predict(X_test).iloc[0]
        bootstrap_probs.append(prob)
    except np.linalg.LinAlgError:
        fail_count += 1
        continue
```

```
In [52]: bootstrap_preds = np.array(bootstrap_probs).flatten()

lower_bound = np.percentile(bootstrap_preds, 2.5)
upper_bound = np.percentile(bootstrap_preds, 97.5)
confidence_interval = (lower_bound, upper_bound)

print(f"95% Confidence Interval at test point:")
print(f"Lower Bound: {lower_bound:.4f}")
print(f"Upper Bound: {upper_bound:.4f}")

plt.figure(figsize=(10, 6))
sns.histplot(bootstrap_preds, bins=30, kde=True, color='skyblue', edgecolor='black')
plt.axvline(lower_bound, color='red', linestyle='--', label=f'2.5th Percentile ({lower_bound:.2f})')
plt.axvline(upper_bound, color='red', linestyle='--', label=f'97.5th Percentile ({upper_bound:.2f})')
plt.title(f'Bootstrap Distribution of Logistic Regression Predictions for Test Point')
plt.xlabel('Predicted Probability of Survival')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

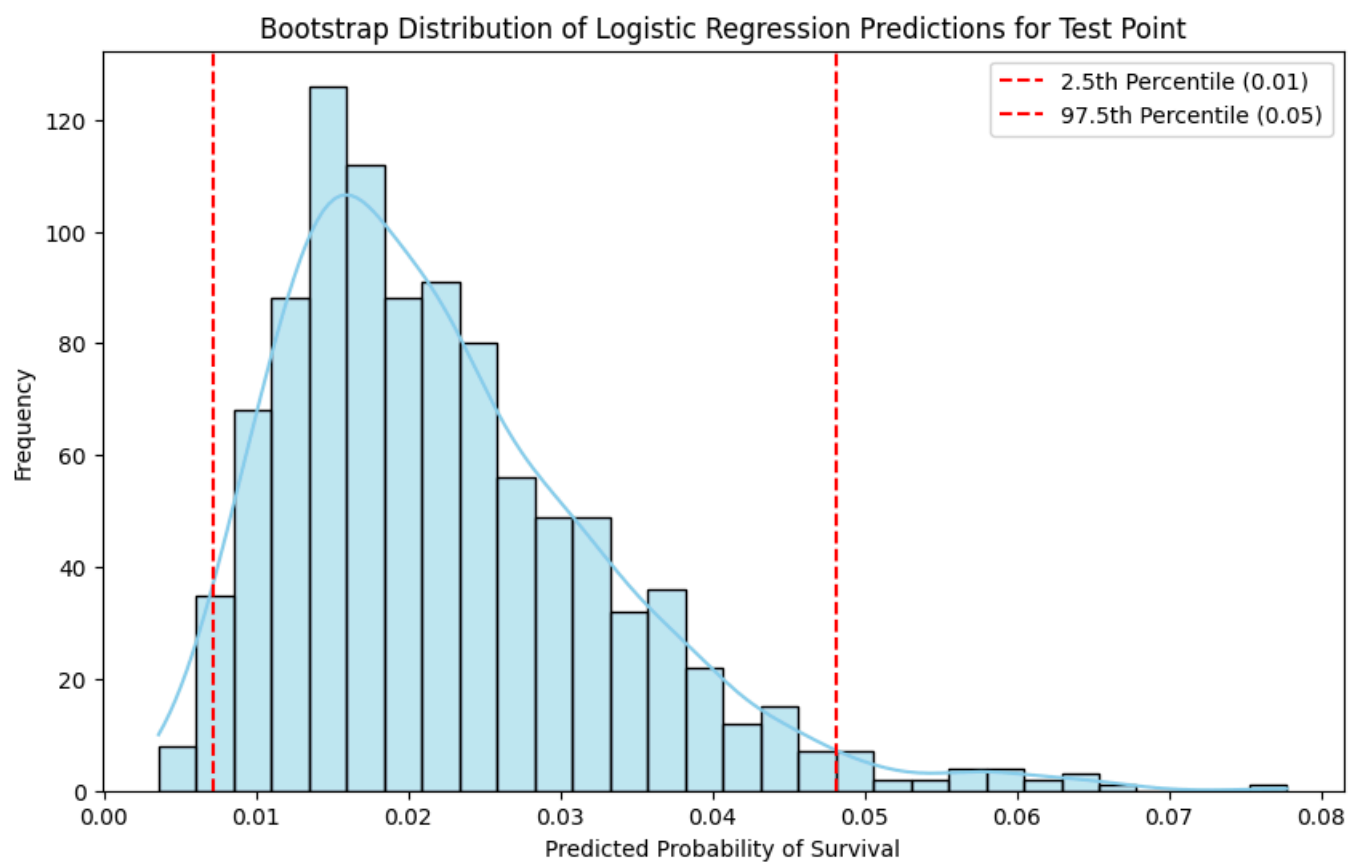
95% Confidence Interval at test point:

Lower Bound: 0.0072

Upper Bound: 0.0482

/Users/mengchuishuo/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```



Problem 4-5

```
In [53]: from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis as QDA

test_point = data_4.iloc[[0]]
train_data = data_4.iloc[1:]

X_train = pd.get_dummies(train_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
X_train = X_train.astype(float)
y_train = train_data['Survived']

X_test = pd.get_dummies(test_point[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
X_test = X_test.astype(float)

qda_model = QDA()
qda_model.fit(X_train, y_train)
```

```
Out[53]: ▼ QuadraticDiscriminantAnalysis
QuadraticDiscriminantAnalysis()
```

```
In [54]: pred_prob = qda_model.predict_proba(X_test)
```

```
In [55]: pred_prob
```

```
Out[55]: array([[0.63698412, 0.36301588]])
```

We can find the probability of this person's survival is much lower than that of death. So the survived predicted is 0.

```
In [56]: n_bootstraps = 1000
bootstrap_probs = []
fail_count = 0

for i in range(n_bootstraps):
    bootstrap_sample = resample(train_data, replace=True, n_samples=len(train_data))

    X_boot = pd.get_dummies(bootstrap_sample[['Pclass', 'Sex', 'Age', 'SibSp', 'Fare']], drop_first=True)
    X_boot = X_boot.reindex(columns=X_train.columns, fill_value=0)
    X_boot = X_boot.astype(float)
    y_boot = bootstrap_sample['Survived']

    if len(np.unique(y_boot)) < 2:
        fail_count += 1
```



```

        continue

    try:
        boot_model = QDA()
        boot_model.fit(X_boot, y_boot)
        prob = boot_model.predict_proba(X_test)[0, 1]
        bootstrap_probs.append(prob)
    except Exception:
        fail_count += 1
        continue

```

```

In [57]: bootstrap_probs = pd.DataFrame(bootstrap_probs)
        pred_interval = bootstrap_probs.quantile([0.025, 0.975])

```

```

lower = pred_interval.iloc[0, 0]
upper = pred_interval.iloc[1, 0]

```

```

print(f"Bootstrap 95% prediction interval: ({lower:.4f}, {upper:.4f})")

```

```

Bootstrap 95% prediction interval: (0.1932, 0.6005)

```

```

In [58]: plt.figure(figsize=(10, 6))
        sns.histplot(bootstrap_probs, bins=30, kde=True, color='skyblue', edgecolor='black')

        plt.axvline(lower, color='red', linestyle='--', label=f'2.5th Percentile ({lower:.2f})')
        plt.axvline(upper, color='red', linestyle='--', label=f'97.5th Percentile ({upper:.2f})')

        plt.title('Bootstrap Distribution of QDA Predictions for Test Point')
        plt.xlabel('Predicted Probability of Survival')
        plt.ylabel('Frequency')
        plt.legend()
        plt.show()

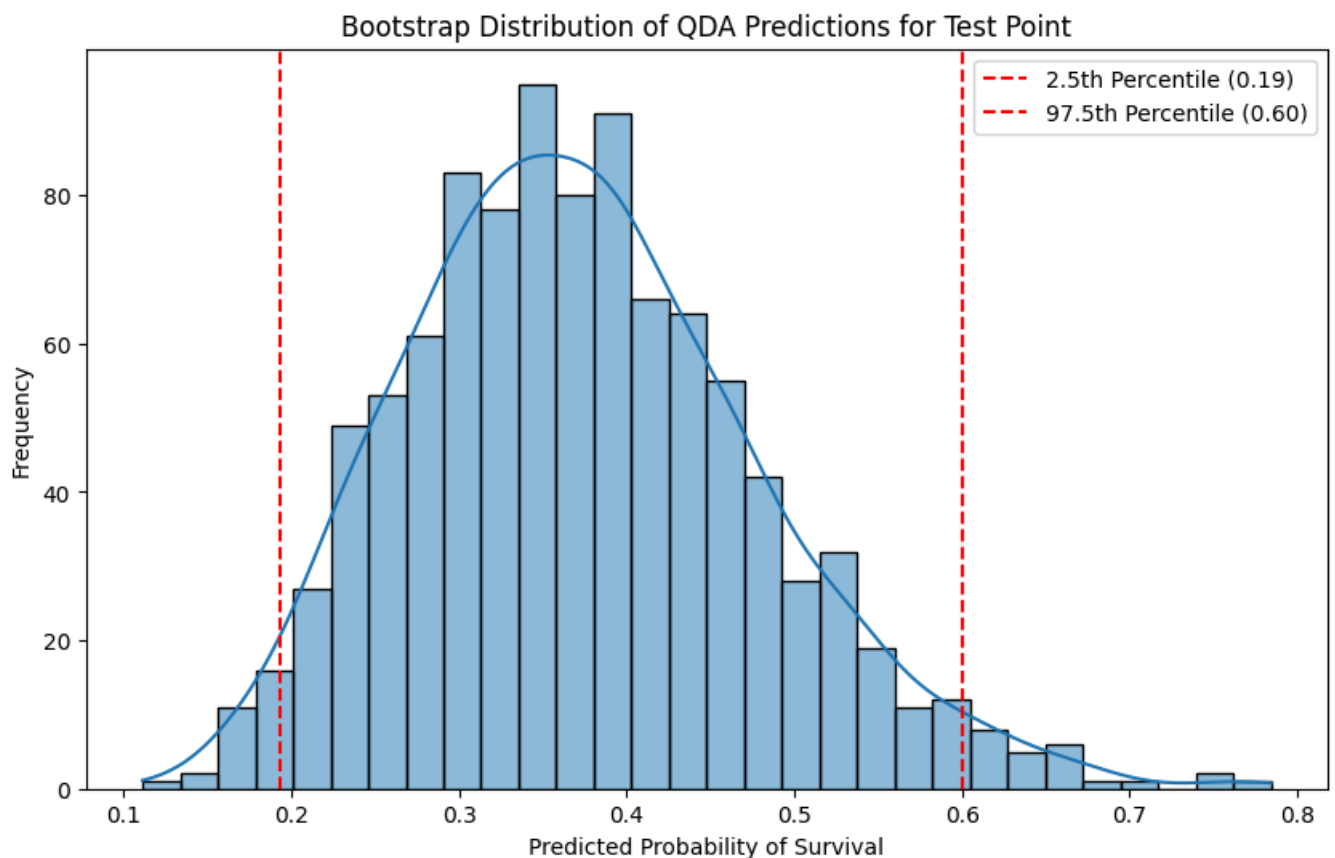
```

/Users/mengchuishuo/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```

with pd.option_context('mode.use_inf_as_na', True):

```



According to Bootstrap distribution, we can find survival rate greater than 0.5 accounts for a small proportion.